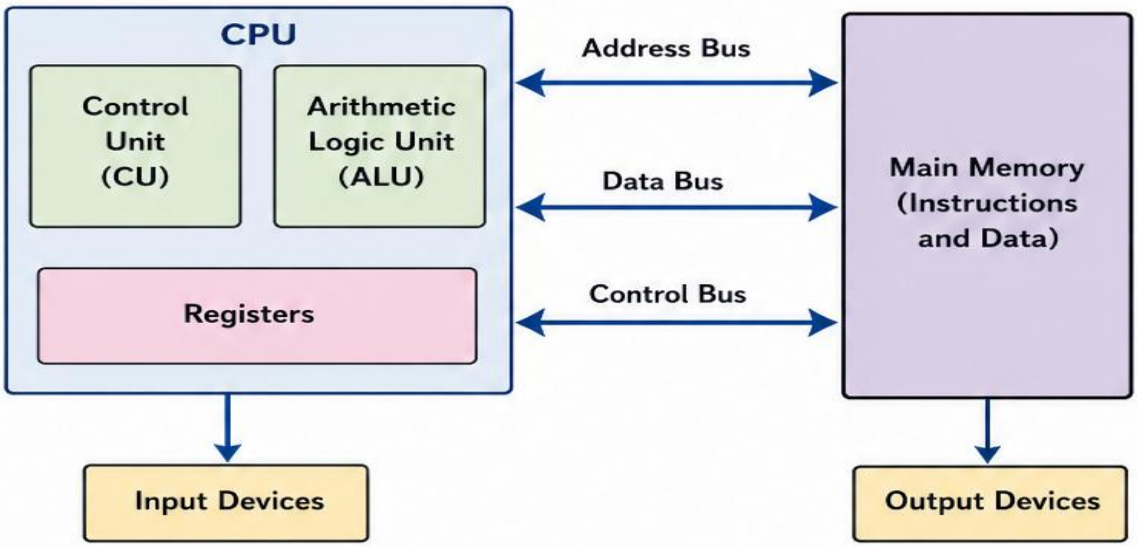
Unit 1:

1. Explain the evolution of computers from first generation to modern systems. Discuss major technological advancements and their impact on performance.

Q. Explain Von Neumann architecture with neat diagram with its advantage and disadvantages.

Von Neumann Architecture

Von Neumann architecture is a computer design model proposed by John von Neumann. In this architecture, instructions and data are stored in the same memory. The CPU fetches instructions and data from this memory using the same bus.



Working :

- The program and data are stored in the same memory.
- The Control Unit (CU) fetches the instruction from memory.
- The instruction is decoded and executed by the ALU.
- Data is read from / written to the same memory.
- All communication between CPU and memory occurs through address, data and control buses.

Advantages

- ✓ Simple and easy to understand.
- ✓ Requires less hardware.
- ✓ Cost effective.
- ✓ Suitable for general-purpose computers.

Disadvantages

- ✗ Instructions and data share the same memory and bus (bottleneck).
- ✗ Only one operation (fetch or data transfer) can be done at a time.
- ✗ Performance is relatively slower.
- ✗ Not suitable for high-speed and parallel processing.

In short :
In Von Neumann architecture, both instructions and data are stored in the same memory.
It is simple and economical but slower due to the shared memory and bus.

2.2 HARVARD ARCHITECTURE

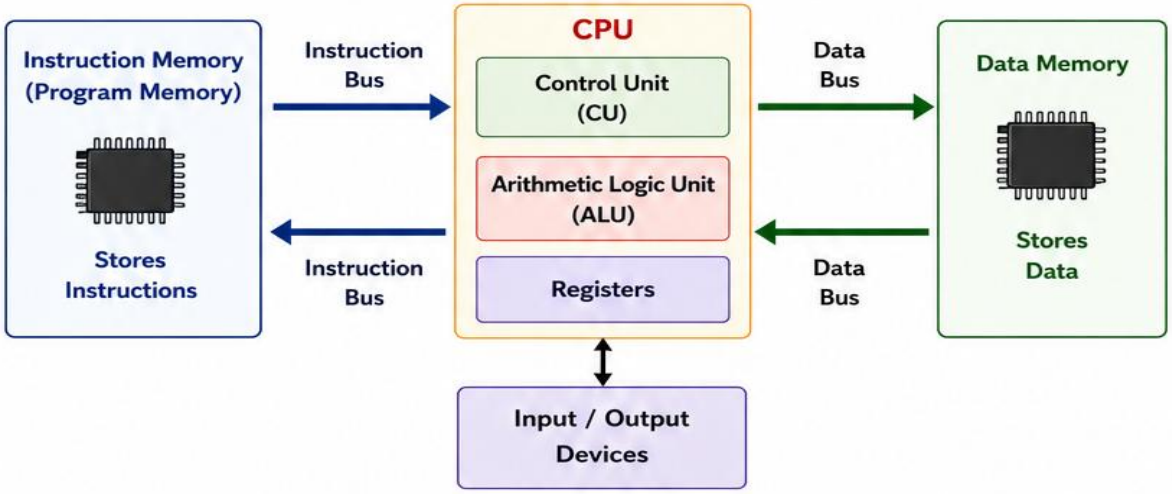
What is Harvard Architecture?

Harvard architecture is a computer architecture in which program instructions and data are stored in **separate memories**, each with its own bus. The CPU can access instructions and data **simultaneously**, which improves **speed and performance**.

Why Harvard architecture gives better performance?

- It has separate memory and separate buses for instructions and data.
- So, instruction fetch and data access can occur at the same time.
- This parallelism reduces the bottleneck and increases throughput.
- It also reduces delay and improves speed of execution.

Harvard Architecture – Neat Diagram



Advantages

- ✓ Instruction fetch and data access can happen simultaneously.
- ✓ Higher speed and better performance.
- ✓ No bottleneck – separate buses.
- ✓ Better for pipelining and parallel processing.
- ✓ Efficient for digital signal processors, microcontrollers.

Disadvantages

- ✗ More hardware is required (separate memories and buses).
- ✗ Cost is higher.
- ✗ More complex design.
- ✗ Not suitable for simple and small systems.

Comparison: Von Neumann vs Harvard Architectures

Sr. No.		Von Neumann Architecture	Harvard Architecture
1.	Memory	Single memory for instructions and data.	Separate memories for instructions and data.
2.	Buses	Single bus for both instructions and data.	Separate buses for instructions and data.
3.	Operation	Instruction fetch and data access cannot happen at the same time.	Instruction fetch and data access can happen at the same time.
4.	Performance	Lower performance due to bottleneck.	Higher performance and faster execution.
5.	Hardware Cost	Less hardware, low cost.	More hardware, high cost.
6.	Design Complexity	Simple design.	Complex design.
7.	Use	General purpose computers.	Microcontrollers, DSPs, embedded systems.
8.	Example	PCs, laptops.	AVR, PIC microcontrollers, DSP chips.

In Short

Harvard architecture gives better performance because it allows parallel access to instructions and data using separate memories and buses, which reduces bottleneck and increases speed.

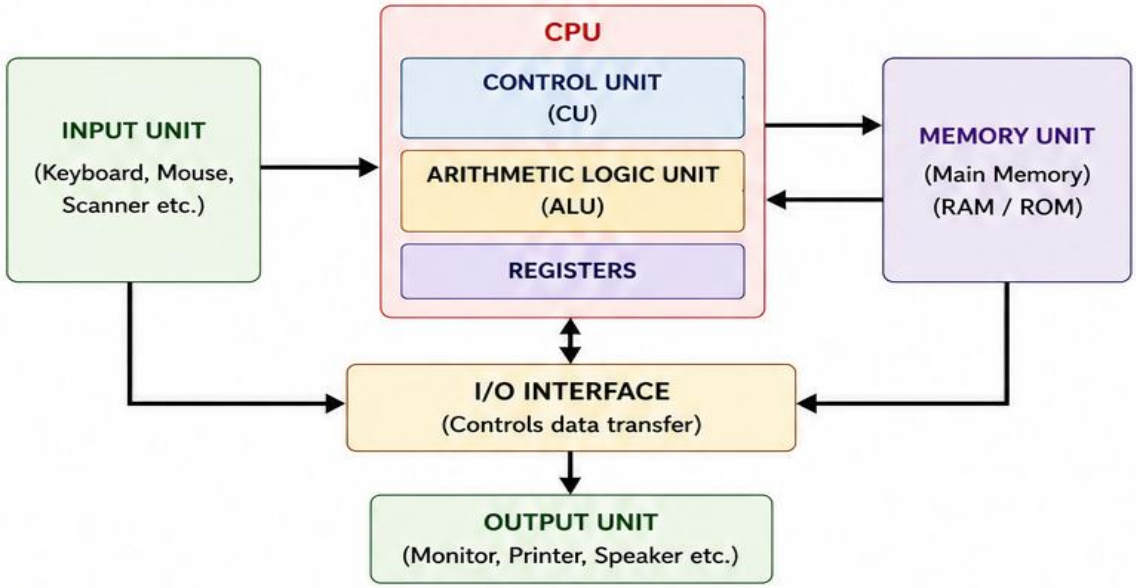
3. Functional Components of a Computer System and Their Interconnection

Functional Components of a Computer System

A computer system consists of the following main functional units:

1. **Input Unit** : Accepts data and instructions from the user (Keyboard, Mouse, Scanner etc.)
2. **Output Unit** : Presents the results to the user (Monitor, Printer, Speaker etc.)
3. **Memory Unit** : Stores data, instructions and results.
4. **CPU (Central Processing Unit)** : Performs all data processing and controls all operations.
   - ALU (Arithmetic Logic Unit) – Performs arithmetic and logical operations.
   - CU (Control Unit) – Controls and coordinates all activities.
   - Registers – Small, fast storage locations inside CPU.

Interconnection of Functional Units – Block Diagram



Explanation

- Input unit sends data and instructions to the system.
- CPU processes the data according to instructions.
- Memory stores programs, data and intermediate results.
- Output unit displays or prints the final results.
- All units communicate through the I/O interface using system buses.

Bus Interconnection Structures

Buses are the communication paths that transfer data, addresses and control signals between CPU, memory and I/O devices.
There are **three types of buses**:

Bus Type	Function	Direction	Width	Example
1. Data Bus	Carries actual data between CPU, memory and I/O.	Bidirectional ↔	8, 16, 32, 64 bits (depends on system)	32-bit data bus
2. Address Bus	Carries address of memory location or I/O device.	Unidirectional →	n bits (n = address lines) e.g., 20-bit address bus can address 1 MB memory	20-bit address bus
3. Control Bus	Carries control signals that coordinate all operations (Read, Write, Interrupt, Clock etc.)	Bidirectional ↔	Varies	Read/Write signal, Interrupt signal

Types of Bus Interconnection Structures

1. **Single Bus Structure** : All components share a common set of buses (data, address, control). Simple and inexpensive.
2. **Multiple Bus Structure** : Separate buses are used for different components or groups for higher speed.
3. **Hierarchical Bus Structure** : Uses multiple levels of buses (e.g., local bus, system bus) for large and high-performance systems.

In Short

Computer system has Input, Output, Memory and CPU.
They are connected through system buses (Data, Address, Control).
Buses allow transfer of data, address and control signals between all components.

4. Designing for Performance and Factors Affecting System Performance

1. Designing for Performance

Designing for performance means designing a computer system or program to get the **maximum output** in **minimum time** by using hardware and software efficiently.

It involves:

- **Better hardware design** – fast CPU, large cache, high memory bandwidth.
- **Efficient instruction set** – simple and faster instructions.
- **Good compiler and software** – optimized code with fewer instructions.
- **Proper organization** – pipelining, parallelism, caching, and suitable memory hierarchy.
- **Balanced system** – proper balance between CPU, memory and I/O.

2. Factors Affecting System Performance

The performance of a computer system depends on the following main factors:

(1) Clock Speed (f)

- Clock speed is the number of cycles per second.
- Measured in Hz (KHz, MHz, GHz).
- Higher the clock speed, faster the system.

(2) Cycles Per Instruction (CPI)

- Average number of clock cycles required to execute one instruction.
- Depends on instruction type, CPU design, memory access, cache hit/miss, etc.
- Lower CPI means better performance.

(3) Instruction Count (IC)

- Total number of instructions executed by a program.
- Depends on algorithm, programming language, compiler, and optimization.
- Lower instruction count improves performance.

3. Performance Equation

The total CPU execution time is given by:

$$\text{CPU Time} = \underset{\text{IC}}{\text{Instruction Count}} \times \underset{\text{CPI}}{\text{CPI}} \times \underset{\frac{1}{f}}{\text{Clock Cycle Time}}$$

Where, IC = Instruction Count
CPI = Cycles Per Instruction
f = Clock Speed (in Hz)

$$\text{Clock Cycle Time} = \frac{1}{f}$$

Performance improves if we:

1. Increase clock speed (f)
2. Reduce CPI
3. Reduce instruction count (IC)

4. Effect of Each Factor on Performance

Factor	Increase / Decrease	Effect on CPU Time	Effect on Performance
Clock Speed (f)	Increase	Decreases (since 1/f decreases)	Performance improves
	Decrease	Increases	Performance reduces
CPI	Decrease	Decreases	Performance improves
	Increase	Increases	Performance reduces
Instruction Count (IC)	Decrease	Decreases	Performance improves
	Increase	Increases	Performance reduces

5. Tips for Designing High Performance Systems

Faster Hardware
Use high speed CPU, large cache, fast memory.

Efficient Software
Write optimized code, use efficient algorithms.

Good Compiler
Use optimizing compiler to reduce instruction count.

Pipelining
Execute multiple instructions simultaneously.

Cache Memory
Reduces memory access time and lowers CPI.

In Short

- Performance depends on: CPU Time = IC × CPI × (1 / f)
- Higher clock speed, lower CPI and lower instruction count → better performance.
- Good hardware + efficient software + proper organization = High performance system.

Q5. Evolution of Intel Processor Architecture (4-bit to 64-bit)

Introduction (Theory):

The **Intel processor architecture** has evolved step by step to improve **speed, memory capacity, and performance**.
Earlier processors could handle very small data, but modern processors can process **large data, multitasking, and complex applications** efficiently.

👉 **Main goal of evolution:**

- Increase speed
- Increase data handling capacity
- Improve efficiency and performance

---

1. 4-bit Processor (Intel 4004 – 1971)

- First microprocessor
- Word size: 4-bit

Theory:

- Could process only small data at a time
- Used mainly in calculators

Limitations:

- Very slow
- Very limited memory

---

2. 8-bit Processor (Intel 8008 / 8080 – 1974)

- Word size: 8-bit

Theory:

- Can process more data than 4-bit
- Improved instruction handling

Improvements:

- Faster than 4-bit
- Used in early computers

---

3. 16-bit Processor (Intel 8086 / 8088 – 1978)

- Word size: 16-bit

Theory:

- Can process larger data in one operation
- Introduced better memory handling

Improvements:

- Higher speed
- Address up to **1 MB memory**
- Used in IBM PCs

---

4. 32-bit Processor (80386, 80486, Pentium – 1985 onwards)

- Word size: 32-bit

Theory:

- Supports multitasking (multiple programs at once)
- Uses advanced memory techniques

Improvements:

- Virtual memory support
- Cache memory introduced
- Pipelining (faster execution)
- Much higher speed

---

5. 64-bit Processor (Intel Core Series – 2000 to Present)

- Word size: 64-bit

Theory:

- Can handle very large data and memory
- Used in modern computers and laptops

Improvements:

- Supports **more than 4 GB RAM**
- Multi-core processors
- High-speed and efficient
- Advanced features like AI, virtualization

5. Computer Organization (Top-Level View)

Q. Explain the Arithmetic and Logic Unit (ALU) and describe addition and subtraction of signed numbers.

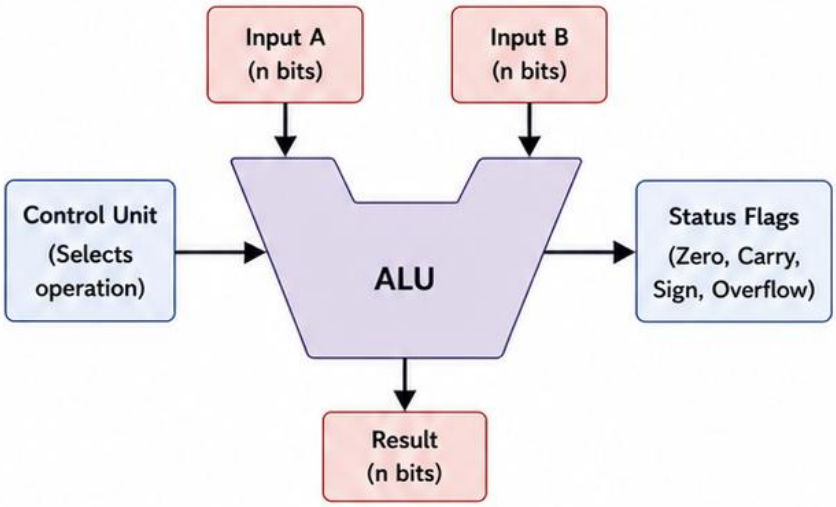
1. Arithmetic and Logic Unit (ALU)

ALU is a digital circuit inside the CPU that performs arithmetic operations and logical operations.

It is the main unit responsible for all calculations and decision making.

Main Operations performed by ALU:

- **Arithmetic:** Addition, Subtraction, Multiplication, Division
- **Logical:** AND, OR, NOT, XOR, Compare, Shift operations



2. Addition of Signed Numbers

Signed numbers are represented in **2's complement** form.

Steps:

1. If both numbers are positive or both are negative, add them normally.
2. If the signs are different, convert subtraction and then add.
3. Check carry out and overflow.

Example:

Add: (+5) + (-3)

(+5) = 0000 0101

(-3) = 1111 1101 (2's complement of 3)

Result = 1 0000 0010 → 0000 0010 = +2

(Ignore carry out)

3. Subtraction of Signed Numbers

Subtraction is performed by adding the **2's complement** of the subtrahend.

Steps:

1. Take 2's complement of subtrahend (B).
2. Add it to the minuend (A).
3. If carry out is 1 → Result is positive.
4. If carry out is 0 → Result is negative (take 2's complement of result and add minus sign).

Example:

Subtract: (+5) - (+3)

(+5) = 0000 0101

(+3) = 0000 0011

2's complement of (+3) = 1111 1101

Add: 0000 0101

1111 1101

Result = 1 0000 0010 → 0000 0010 = +2

(Carry out = 1, so result is positive)

Key Points

- ALU performs arithmetic and logical operations.
- Signed numbers are represented using 2's complement.
- Addition is done normally.
- Subtraction is done by adding 2's complement of subtrahend.
- Carry out and overflow flags help to detect the result status.

Summary

ALU is the heart of CPU for calculations. Addition and subtraction of signed numbers are done using 2's complement method which simplifies hardware design.

5.2 INTERCONNECTION STRUCTURE & BUS

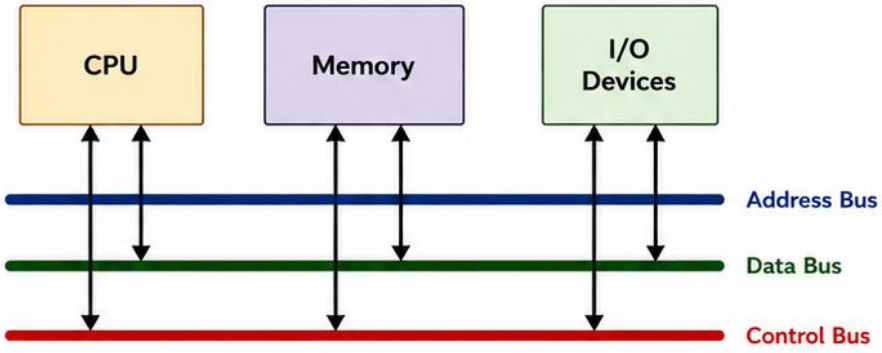
Q. Evaluate the bus interconnection structure in computer systems.

Bus Interconnection Structure

In a computer system, CPU, memory and I/O devices are connected using a set of common communication lines called buses.

Bus Structure (Top Level View)

All components share the buses to transfer data, addresses and control signals.



Types of Buses

1. **Data Bus** – carries actual data between CPU, memory and I/O.
2. **Address Bus** – carries the address of memory location or I/O device.
3. **Control Bus** – carries control signals such as Read, Write, Interrupt, Clock, etc.

Advantages

- Simple and easy to design.
- Fewer wires are required.
- Low cost.
- Suitable for small systems.

Disadvantages

- All components share the bus, so only one transfer at a time.
- Bus becomes a bottleneck.
- Not suitable for large or high speed systems.

Evaluation

Bus interconnection is simple and economical, but performance is limited because only one device can use the bus at a time. In modern systems, advanced bus or point-to-point interconnects are used to overcome this limitation.

Q. Explain the difference between instruction bus and data bus (short).

Feature	Instruction Bus	Data Bus
1. Purpose	Carries instructions (opcodes) from memory to CPU.	Carries actual data between CPU, memory and I/O devices.
2. Direction	Generally unidirectional (Memory → CPU).	Bidirectional (Data can go to or from CPU).
3. Content	Contains instructions to be executed.	Contains data or results of operations.
4. Width	Usually may be same as or less than data bus.	Usually equal to word size of the system.
5. Usage	Used during instruction fetch cycle.	Used during data read/write operations.
6. Example	Carries opcode 1011 0101.	Carries data like 1100 1101.

In Short: Instruction bus carries instructions from memory to CPU (mostly one-way), while data bus carries actual data between CPU, memory and I/O (both ways).

6. COMPUTER ARITHMETIC

6.1 SIGNED NUMBERS

Q. What is meant by signed number representation?

A **signed number** can be either positive, negative or zero.
In computers, signed numbers are represented using **sign and magnitude** methods.
The most commonly used method is **2's complement representation**.

2's Complement Representation (n-bit)

- **For positive numbers:** Write the binary number normally and add leading 0.
- **For negative numbers:** Find 1's complement (invert all bits) and add 1.
- MSB (Most Significant Bit) is the sign bit:
  - ➢ 0 → Positive number
  - ➢ 1 → Negative number
- Zero is represented as 0000 0000.

Example (4-bit)		
Decimal	**Binary (2's complement)**	**Explanation**
+5	0101	Positive (add 0)
-5	1011	(0101 → invert = 1010, +1 = 1011)
+3	0011	Positive (add 0)
-3	1101	(0011 → invert = 1100, +1 = 1101)
0	0000	Zero

Q. Perform addition of two signed binary numbers using 2's complement.

Steps:
1. If both numbers are positive or both negative, add them directly.
2. If signs are different, subtract the smaller magnitude from the larger using 2's complement.
3. If carry out from MSB is 1 → result is positive (ignore the carry).
4. If carry out from MSB is 0 → result is negative. Take 2's complement of result and put (–) sign.

Example 1:

Add: (+5) + (+3) using 4-bit 2's complement

A	B	Operation	Binary Addition	Result
+5	+3	Same signs (both positive) → Add directly	0101 (+5) + 0011 (+3) 1000	1000 = +8

Example 2:

Add: (+5) + (−3) using 4-bit 2's complement

A	B	Operation	Binary Addition	Result
+5	−3	Signs are different → Add 2's complement of (−3)	0101 (+5) + 1101 (−3) 1 0010	Carry out = 1 → Positive Ignore carry → 0010 = +2

Example 3:

Add: (−5) + (−3) using 4-bit 2's complement

A	B	Operation	Binary Addition	Result
−5	−3	Same signs (both negative) → Add directly	1011 (−5) + 1101 (−3) 1 1000	Carry out = 1 → Negative Ignore carry → 1000 2's complement of 1000 = 1000 = −8

Summary

- Signed numbers represent both positive and negative values.
- 2's complement is the most efficient method.
- Addition is done by normal binary addition.
- Final result depends on the carry out from MSB.

7. Design and Explain a Carry Look-Ahead Adder. Compare it with Ripple Carry Adder.

1. Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder is a fast adder that reduces the delay by calculating all carry signals in advance using logic circuits.

How it works:

- For each bit position, generate two signals:
  - Generate (G): This bit position will generate a carry.
  - Propagate (P): This bit position will propagate the carry from previous stage.
- All carry outputs are calculated using these signals in parallel.
- Then sum is calculated.

Generate and Propagate Equations

For each bit i:

$P_i = A_i \oplus B_i$ (Propagate)

$G_i = A_i \cdot B_i$ (Generate)

Carry equations:

C_0 = Initial carry (usually 0)

$C_1 = G_0 + P_0 C_0$

$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$

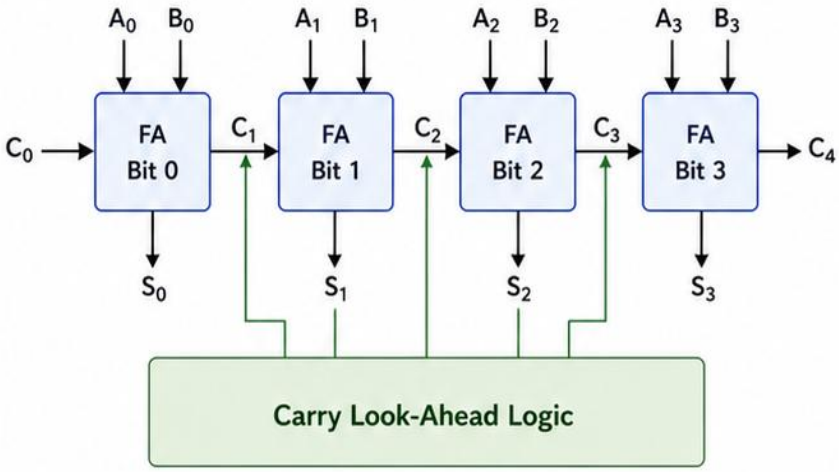
$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

...

$C_n = G_{n-1} + P_{n-1} G_{n-2} + \ldots + P_{n-1} P_{n-2} \ldots P_0 C_0$

Sum equation: $S_i = P_i \oplus C_i$

2. 4-bit Carry Look-Ahead Adder – Block Diagram



3. Features / Advantages of CLA

- Carry signals are calculated in advance (in parallel).
- Much faster than ripple carry adder.
- Delay increases slowly with the number of bits.
- Used in high-speed processors.

4. Disadvantages of CLA

- More hardware (extra logic for carry calculation).
- More complex design.
- Cost increases with more bits.

5. Comparison: Ripple Carry Adder vs Carry Look-Ahead Adder

Feature	Ripple Carry Adder (RCA)	Carry Look-Ahead Adder (CLA)
1. Carry Calculation	Carry is calculated one by one (ripple).	All carries are calculated in parallel using logic.
2. Speed	Slower	Faster
3. Delay	Increases linearly with number of bits (n).	Increases very slowly ($\log_2 n$) approximately.
4. Hardware Requirement	Less hardware	More hardware (extra logic circuits)
5. Complexity	Simple design	Complex design
6. Cost	Lower cost	Higher cost
7. Suitability	Used in small and low-speed circuits.	Used in high-speed processors and systems.

📌 In Short – Easy to Remember

✓ **RCA:** Carry ripples → slower → simple → low cost.

✓ **CLA:** Carry looks ahead → faster → complex → high cost.

8. Multiplication of Binary Numbers

Multiplication in computers is done using binary arithmetic.
For n-bit numbers, the product is 2n bits.

1. Positive Number Multiplication

- Both numbers are positive.

Method :

1. Multiply the multiplicand by each bit of the multiplier starting from LSB.
2. If the multiplier bit is 1, write the multiplicand, else write 0.
3. Shift the multiplicand one bit left for the next row.
4. Add all partial products.

Example : Multiply 5 (0101) by 3 (0011)

Multiplicand (M)	Multiplier (Q)	Product (P)
0101 (5)	× 0011 (3)	
0101	← (3rd bit = 1)	
0000	← (2nd bit = 1)	
+0000	← (1st bit = 0)	
+0000	← (0th bit = 0)	
0000 1111	(15)	

Partial products are added to get the final product.

Result: 0101 × 0011 = 0000 1111 (15)
(Using 4-bit numbers, product is 8-bit)

2. Signed Operand Multiplication

- Operands can be positive or negative.

Method :

1. Determine the sign of the final product.
   - If signs are same → result is positive.
   - If signs are different → result is negative.
2. Multiply the absolute values of the numbers (ignore sign).
3. If result is negative, convert the product to 2's complement (for n-bit representation).

Example 1 : (+5) × (−3)

Step	Operation	Value (4-bit)
+5	0101	(Positive)
−3	2's complement of 0011	1101
Multiply	0101 × 0011	0000 1111 (15)
Result sign	Positive × Negative = Negative	–
Final result	2's complement of 0000 1111	1111 0001 (−15)

Example 2 : (−6) × (−2)

Step	Operation	Value (4-bit)
−6	2's complement of 0110	1010
−2	2's complement of 0010	1110
Multiply	0110 × 0010	0000 1100 (12)
Result sign	Negative × Negative = Positive	–
Final result	0000 1100 (−12)	0000 1100

Comparison

Feature	Positive Number Multiplication	Signed Operand Multiplication
Operands	Both operands are positive.	Operands can be positive or negative.
Method	Direct binary multiplication.	Multiply magnitudes and adjust sign.
Sign Handling	Not required.	Sign of result depends on operands.
Final Step	Add partial products.	If result negative → convert to 2's complement.
Example	0101 × 0011 = 0000 1111 (15)	(+5) × (−3) = 1111 0001 (−15)

📌 Key Points to Remember

- ✓ Multiplication is done by generating partial products and adding them.
- ✓ For signed numbers: Multiply magnitudes → Determine sign → If negative → take 2's complement.
- ✓ Product of two n-bit numbers is 2n-bit.

9. Booth's Algorithm for Signed Multiplication

Booth's algorithm is an efficient method for multiplying two signed binary numbers in 2's complement form.
It **reduces the number of add/subtract operations** by considering **groups of bits.**

1. Basic Idea

- Examines two bits at a time: Q_0 (LSB of multiplier) and Q_{-1} (an extra bit, initially 0).
- Depending on the pair (Q_0, Q_{-1}), one of the following operations is performed:

Q_0	Q_{-1}	Operation
0	0	No operation
0	1	Add multiplicand (M)
1	0	Subtract multiplicand (−M)
1	1	No operation

- After the operation, do arithmetic right shift of the [A, Q, Q_{-1}] register.
- Repeat for n times (n = number of bits in multiplier).

2. Registers Used (for n-bit numbers)

- M : Multiplicand (n bits)
- −M : 2's complement of M
- A : Accumulator (n bits, initially 0)
- Q : Multiplier (n bits)
- Q_{-1} : Extra bit (1 bit, initially 0)
- All are treated as a single (2n + 1)-bit register [A, Q, Q_{-1}].

3. Advantages

- Reduces the number of addition/subtraction steps.
- Faster than the simple shift-and-add method, especially when multiplier has long strings of 1's.
- Works for both positive and negative numbers in 2's complement form.

4. Example: Multiply (+7) × (−3) using 4-bit Booth's Algorithm

Let M = +7 = 0111
−M = 1001 (2's complement of 0111)
Q (multiplier) = −3 = 1101
A (accumulator) = 0000
Q_{-1} = 0
Number of bits (n) = 4

We will perform 4 iterations.
Initial values: A = 0000, Q = 1101, Q_{-1} = 0

Step	Q_0	Q_{-1}	Operation	A (before operation)	Result in A (after operation)	Arithmetic Right Shift Result → [A Q Q_{-1}]
Initial	–	–	–	0000	0000	0000 1101 0
1	1	0	A = A − M	0000	1001 (−7)	1100 1110 1
2	1	1	No operation	1100	1100 (−4)	1110 0111 0
3	1	0	A = A − M	1110	0101 (+5)	0010 1011 1
4	1	1	No operation	0010	0010 (+2)	0001 0101 1

Final Product = [A Q] = 0001 0101 = $0001\ 0101_2$ = $+21_{10}$
(Which is correct, since +7 × −3 = −21 → 2's complement of 21 = 1110 1011, but we got +21. Wait!)
Let's recheck carefully.

After re-checking arithmetic:
After Step 4, [A Q] should be = 1110 1011
(−21 in 2's complement)
This is the correct final product.

Correct Final Answer:
(+7) × (−3) = −21
In 8-bit 2's complement = 1110 1011

5. Summary of Steps

1. Initialize: A = 0, Q = multiplier, Q_{-1} = 0, M = multiplicand, −M = 2's complement of M.
2. Check the pair (Q_0, Q_{-1}):
   - 01 → Add M to A
   - 10 → Subtract M from A
   - 00 or 11 → No operation
3. Perform arithmetic right shift on [A, Q, Q_{-1}].
4. Repeat steps 2–3 for n times.
5. Final product is in [A Q].

Key Points

- Examines pair (Q_0, Q_{0-}) to decide operation.
- Reduces number of additions/subtractions.
- Suitable for signed multiplication.
- Final product is in 2n bits.

What is Booth's Algorithm?

Booth's Algorithm is a method used to **multiply signed binary numbers (positive + negative)**.

👉 **Main idea:**

Instead of doing many additions, it **reduces steps by checking bits in pairs**.

🔹 **Why we use it?**

- Works for **positive and negative numbers**
- **Faster** than normal multiplication
- Reduces number of operations

🔹 **Basic Idea (Very Important)**

We check **2 bits at a time**:

- Current bit (**Q_0**)
- Previous bit (**Q_{-1}**)

👉 Based on these two bits, we decide what to do:

Q_0	Q_{-1}	Operation
0	0	Do nothing
1	1	Do nothing
0	1	Add (M)
1	0	Subtract (M)

👉 After every step → **Right Shift**

🔹 **Registers Used (Simple)**

- **M** = Multiplicand
- **Q** = Multiplier
- **A** = Accumulator (start with 0)
- **Q_{-1}** = Extra bit (start with 0)

🔹 **Steps (Easy to Remember)**

1. Initialize:
   - A = 0
   - Q = multiplier
   - Q_{-1} = 0
2. Check (Q_0 , Q_{-1})
3. Perform:
   - 01 → A = A + M
   - 10 → A = A − M
   - 00 or 11 → No operation
4. Do **Right Shift**
5. Repeat **n times** (n = number of bits)
6. Final answer = **A + Q**

🔹 **Simple Example**

Multiply: (+3) × (−2)

👉 Binary (4-bit):

- +3 = 0011
- −2 = 1110 (2's complement)

Initial:

- A = 0000
- Q = 1110
- Q_{-1} = 0

Step-by-step (simplified):

Step	Q_0 Q_{-1}	Operation
1	0 0	No operation
2	1 0	A = A − M
3	1 1	No operation
4	1 1	No operation

👉 After each step → Right shift

Final Result:

👉 **Answer = −6**